Nama : I Wayan Astawa Putra

NIM : 2415354028

BAGIAN A

Data Count	XML (Bytes)	JSON (Bytes)	Reduksi Efisiensi (%)
10	282	1166	-313.48%
50	722	5826	-706.93%
100	1273	11652	-815.32%
500	5673	58652	-933.88%

BAGIAN B

1. Analisis Struktur Payload
   Jika XML :
   - Setiap elemen memerlukan tag pembuka dan tag penutup yang berulang
   - Membutuhkan pasangan karakter penutup sudut
   - Membutuhkan tag pembungkus Tingkat atas

   Jika JSON :
   - Hanya menggunakan nama kunci sekali diikuti dengan tanda titik dua dan nilainya
   - Menggunakan struktur yang jauh lebih singkat
   - Cukup dibungkus oleh objek
2. Analisis Deserialisasi / Parsing Sisi Klien
   - **Pemrosesan Respons XML:**

1. **Konversi Teks ke DOM:** Peramban harus membaca respons teks XML dan mengubahnya menjadi *DOM Document Object* menggunakan parser khusus seperti DOMParser (parseFromString(xmlString, "text/xml")).
2. **Navigasi Tree/Nodes:** Klien harus melakukan ekstraksi data menggunakan metode perelusuran pohon DOM seperti getElementsByTagName(), querySelector(), atau XPath.
3. **Konversi Tipe Data Manual:** Semua nilai pada XML dibaca sebagai tipe teks (*string*). Klien harus melakukan *casting* tipe data secara manual (misalnya mengonversi teks angka ke Integer atau Boolean).
   - **Pemrosesan Respons JSON:**

1. **Direct Native Parsing:** Peramban memproses teks JSON menggunakan fungsi bawaan *engine* JavaScript, yaitu JSON.parse(jsonString) (atau otomatis di-parse oleh API modern seperti fetch().json()).
2. **Pengaksesan Objek Langsung:** Data hasil *parsing* langsung menjadi objek/array JavaScript (*native object*) dan dapat diakses secara implisit menggunakan *dot notation* (misal: response.data[0].nama).

3. **Preservasi Tipe Data:** JSON mendukung tipe data bawaan seperti *number*, *boolean*, *array*, *object*, dan *null*, sehingga tidak memerlukan konversi tipe data tambahan di sisi klien.

3. Bottleneck Arsitektur RESTful Synchronous
   Jika sistem legacy membutuhkan waktu **1,5 detik** untuk melakukan *file locking* saat menulis ke data.txt dan terdapat **100 klien** yang memanggil POST /api/v1/students secara bersamaan:

1. **Thread Exhaustion (Kehabisan Sumber Daya Server):**
   - Karena komunikasi bersifat **synchronous blocking** pada HTTP/1.1, setiap permintaan POST akan menahan satu *worker thread/process* server Flask selama minimal 1,5 detik.
   - Dengan 100 permintaan bersamaan, permintaan ke-100 harus menunggu antrean pemprosesan hingga puluhan/ratusan detik (misal: 100 × 1,5 detik jika diproses secara sekuensial pada satu berkas data.txt).
2. **Connection Timeout & Cascading Failure:**
   - Antrean koneksi akan dengan cepat memicu *request timeout* (HTTP 504 Gateway Timeout) pada sisi klien/proxy.
   - Server API Gateway akan mengalami penumpukan memori dan pemblokiran koneksi (*resource exhaustion*), yang berdampak pada gagalnya penanganan permintaan GET (layanan menjadi tidak responsif / *freeze*).